

Security Policy Draft — pqc_crypto_module v0.1.0

This document is a DRAFT aligned with FIPS 140-3 Security Policy structure. It is NOT a certified security policy. It is prepared for future review by an accredited FIPS validation lab.

1. Module name

pqc_crypto_module v0.1.0

2. Cryptographic boundary

All cryptographic operations are isolated within the pqc_crypto_module crate. The DLT application code accesses cryptography exclusively through pqc_crypto_module::api.

Files within boundary: src/api.rs, src/mldsa.rs, src/mlkem.rs, src/hashing.rs, src/rng.rs, src/self_tests.rs, src/approved_mode.rs, src/types.rs, src/errors.rs.

3. Approved algorithms

Algorithm	Standard	Key sizes	Output sizes
ML-DSA-65	FIPS 204	PK: 1952 B, SK: 4032 B	Sig: 3309 B
ML-KEM-768	FIPS 203	PK: 1184 B, SK: 2400 B, CT: 1088 B	SS: 32 B
SHA3-256	FIPS 202	N/A	32 B

4. Non-approved algorithms excluded

Ed25519, ECDSA, RSA, SHA-1, SHA-256, MD5, HMAC-SHA256 are not available through the module API.

5. Roles and services

- **Crypto Officer:** initializes approved mode via `initialize_approved_mode()`
- **User:** calls signing, verification, hashing, encapsulation, decapsulation
- All roles require the module to be in Approved state

6. Key lifecycle

- **Generation:** `generate_mldsa_keypair()`, `generate_mlkem_keypair()`
- **Usage:** `sign_message()`, `verify_signature()`, `mlkem_encapsulate()`, `mlkem_decapsulate()`
- **Zeroization:** Private keys (`MldsaPrivateKey`, `MlKemPrivateKey`) and shared secrets (`MlKemSharedSecret`) implement `ZeroizeOnDrop` — key material is overwritten on drop

7. Self-test behavior

At startup, `initialize_approved_mode()` runs Known Answer Tests for: 1. SHA3-256 (empty string digest comparison) 2. ML-DSA-65 (sign → verify → corrupt → reject) 3. ML-KEM-768 (keygen → encaps → decaps) 4. RNG (continuous random number generator test)

If any test fails, module transitions to Error state. All operations are rejected.

8. Error state behavior

Once in Error state, the module cannot be re-initialized. All cryptographic operations return `CryptoError::ModuleInErrorState`. The node must be restarted.

9. Zeroization behavior

All private key types derive `ZeroizeOnDrop`. When the containing variable goes out of scope, the memory is overwritten with zeros before deallocation.

10. Non-Approved Algorithm Enforcement

Legacy algorithms (Ed25519, SHA-256, HMAC-SHA256) are present only for backward compatibility with pre-PQC blocks.

When the module is in Approved mode: - All non-approved algorithms are disabled via runtime guards (`ensure_not_approved()`) - Any attempt to use legacy guarded functions returns `CryptoError::NonApprovedAlgorithm` - No fallback from approved to non-approved algorithms occurs - The approved-only Cargo feature excludes the legacy module entirely at compile time

These algorithms are outside the approved cryptographic boundary.

11. Future validation notes

- ~~ML-KEM-768 placeholder~~: RESOLVED — implemented via `pqcrypto-mlkem` v0.1.1 with shared secret roundtrip verification.
- The RNG wraps `OsRng` (OS-backed CSPRNG). A DRBG with health tests may be required for full FIPS 140-3 compliance.
- No physical security boundary is defined (software module only).
- Algorithm agility is intentionally limited to approved algorithms within this module.